

Documento Técnico: APIs de HeartCoin

Introducción

Este documento describe y analiza de forma exhaustiva la superficie de API del proyecto HeartCoin: el conjunto completo de endpoints, mecanismos de comunicación y contratos de datos mediante los cuales la aplicación cliente —y, más recientemente, sistemas externos como Original Auth y Original Pay— interactúan con el backend del sistema.

A diferencia de un proyecto con una API REST diseñada y documentada de forma centralizada (por ejemplo, mediante una especificación OpenAPI), la superficie de API de HeartCoin es en gran medida **autogenerada por la plataforma Supabase** a partir del esquema de base de datos, complementada por un conjunto reducido de endpoints verdaderamente personalizados (Edge Functions). Este documento consolida ambos tipos de superficie en un catálogo único y analizado.

Objetivo

Documentar de forma completa y verificable la totalidad de los endpoints del sistema HeartCoin —autogenerados y personalizados—, sus contratos de entrada y salida, sus mecanismos de autenticación y autorización, y evaluar la coherencia, seguridad y madurez de esta superficie de API en su conjunto.

Alcance

Este documento cubre los cinco mecanismos de comunicación disponibles (PostgREST, RPC, Realtime, Edge Functions, Auth), el catálogo completo de las 20 tablas expuestas, las 3 funciones RPC `SECURITY DEFINER`, y las 8 Edge Functions del proyecto (incluyendo las integraciones con MediaAAS, Original Auth y Original Pay). No cubre el detalle exhaustivo de cada política de RLS fila por fila (referido a `ENDPOINTS.md` como fuente primaria) ni la API interna de los servicios de terceros más allá de su superficie de integración con HeartCoin.

Contexto General

HeartCoin no cuenta con un backend de aplicación propio: **Supabase actúa como la totalidad del backend**, generando automáticamente un endpoint REST por cada tabla del esquema (PostgREST), exponiendo funciones de base de datos como endpoints invocables (RPC), y ofreciendo un canal de comunicación en tiempo real (Realtime) sobre el mismo mecanismo. Los únicos endpoints diseñados explícitamente por el equipo del proyecto son las **Edge Functions**, reservadas para las operaciones que requieren credenciales privilegiadas o comunicación con servicios externos.

Metodología

La información de este documento se obtuvo mediante inspección directa del código fuente de cada Edge Function, consulta directa al catálogo de PostgreSQL para determinar las políticas de RLS activas por tabla, y verificación funcional real de los endpoints de integración externa (Original Auth, Original Pay) mediante peticiones firmadas ejecutadas contra el entorno de producción.

Desarrollo del Análisis

1. Los Cinco Mecanismos de la API

Mecanismo	Endpoint base	Naturaleza	Autogenerado o personalizado
Auth	/auth/v1/*	Gestión de sesión y credenciales	Autogenerado (Supabase Auth)
PostgREST	/rest/v1/<tabla>	CRUD directo sobre cada tabla, sujeto a RLS	Autogenerado a partir del esquema
RPC	/rest/v1/rpc/<función>	Invocación de funciones SECURITY DEFINER	Autogenerado a partir de funciones definidas por el equipo
Realtime	WebSocket	Suscripción en vivo a cambios de tabla	Autogenerado, requiere habilitación explícita por tabla
Edge Functions	/functions/v1/<nombre>	Lógica de servidor personalizada	100% personalizado por el equipo

Esta clasificación es relevante porque determina el nivel de control que el equipo tiene sobre cada endpoint: los tres primeros mecanismos derivan su comportamiento del esquema de base de datos y sus políticas (control **declarativo**), mientras que las Edge Functions son código imperativo escrito explícitamente para cada caso de uso.

2. Autenticación de la API

El cliente Flutter se autentica ante Supabase mediante `AuthService`, que envuelve `signUp`, `signInWithPassword`, `signOut` y, más recientemente, el flujo de recuperación de contraseña de tres pasos (`resetPasswordForEmail` → `verifyOTP` → `updateUser`). El token de sesión resultante (`access_token`) se adjunta automáticamente por el SDK en cada llamada subsecuente a PostgREST, RPC y Edge Functions, sin intervención manual del desarrollador.

Las Edge Functions que requieren identificar al usuario (`upload-media`, `delete-account`, las de Original Auth) validan explícitamente ese token mediante un cliente anon antes de proceder — es decir, **no confían únicamente en la verificación de JWT de la plataforma**, sino que revalidan la identidad dentro de su propia lógica, una práctica de defensa en profundidad.

3. Catálogo de Tablas Expuestas (PostgREST)

Las 20 tablas del esquema se agrupan, a efectos de este análisis, en cuatro categorías según su patrón de exposición:

- **Perfiles** (`personal_profiles`, `organization_profiles`, `company_profiles`): lectura restringida por propiedad o visibilidad pública condicionada (`is_public`); escritura exclusiva del propio dueño.
- **Contenido social** (`publicaciones`, `comments`, `post_likes`, `comment_likes`, `follows`, `blocked_users`, `notifications`, `saved_items`): mayoritariamente lectura pública, escritura restringida al propio autor/actor.
- **Participación cívica y campañas** (`iniciativas`, `iniciativa_votes`, `iniciativa_checkins`, `votaciones`, `votacion_votes`, `votacion_comments`, `proveedores`): lectura pública condicionada por estado; varias sin policy de escritura directa para el cliente (`iniciativa_checkins`, `proveedores`, `votaciones`), forzando el uso de RPC o siembra administrativa.
- **Economía y marketplace** (`hc_transactions`, `beneficios`, `beneficio_redemptions`, `servicios`, `servicio_redemptions`): el patrón más restrictivo del esquema — `hc_transactions` y las tablas de canje **no tienen policy de INSERT para ningún cliente**, sin excepción.

Esta gradación —desde tablas de lectura/escritura casi libre hasta tablas completamente cerradas al cliente— refleja una correspondencia directa entre la sensibilidad de negocio del dato y el nivel de restricción aplicado, consistente en la totalidad del esquema.

4. Funciones RPC

Tres funciones constituyen el único camino de escritura para las reglas de negocio que RLS no puede expresar declarativamente:

Función	Endpoint	Complejidad de validación
<code>checkin_iniciativa</code>	POST <code>/rest/v1/rpc/checkin_iniciativa</code>	Alta: geolocalización (haversine) + ventana temporal
<code>redeem_beneficio</code>	POST <code>/rest/v1/rpc/redeem_beneficio</code>	Media: estado, límite de canjes, suficiencia de saldo, bloqueo de fila
<code>redeem_servicio</code>	POST <code>/rest/v1/rpc/redeem_servicio</code>	Media: idéntica a la anterior sobre otro catálogo

Las tres exponen un catálogo de errores nombrados y consistentes (`fuera_de_horario`, `fuera_de_rango`, `hc_insuficiente`, `limite_alcanzado`, entre otros), mapeados a mensajes legibles del lado del cliente — un patrón de contrato de error explícito, preferible a la propagación de mensajes de excepción de base de datos sin traducir.

5. Edge Functions: Catálogo Completo

Función	Dirección	Propósito	Autenticación
<code>upload-media</code>	Cliente → HeartCoin → MediaAAS	Proxy de subida de archivos	Sesión de Supabase (usuario) + HMAC hacia MediaAAS
<code>delete-account</code>	Cliente → HeartCoin	Eliminación de cuenta	Sesión de Supabase (usuario) + clave de servicio
<code>originalauth-register</code>	Cliente → HeartCoin → Original Auth	Dispara verificación de correo en el registro	Sesión de Supabase (usuario) + HMAC hacia Original Auth
<code>originalauth-verify-email</code>	Cliente → HeartCoin → Original Auth	Confirma el código de verificación	Sesión de Supabase (usuario) + HMAC hacia Original Auth
<code>originalauth-resend-verification</code>	Cliente → HeartCoin → Original Auth	Reenvía el código	Sesión de Supabase (usuario) + HMAC hacia Original Auth
<code>originalpay-balance</code>	Original Pay → HeartCoin	Consulta de saldo de HC	HMAC entrante (API key propia de HeartCoin)
<code>originalpay-redeem</code>	Original Pay → HeartCoin	Registro de cobro con HC	HMAC entrante (API key propia de HeartCoin)
<code>originalpay-webhook</code>	Original Pay → HeartCoin	Notificación asíncrona de estado de pago	Credenciales en el body (<code>token_app/secret_key</code>), sin HMAC

Se observan **tres direcciones de flujo distintas** dentro del mismo mecanismo de Edge Functions: (a) cliente hacia HeartCoin exclusivamente, (b) cliente hacia HeartCoin y de ahí hacia un tercero, y (c) un tercero directamente hacia HeartCoin sin intervención del cliente móvil. Esta diversidad es manejada de forma consistente mediante el mismo patrón de aislamiento de credenciales, aunque con mecanismos de autenticación distintos según lo que cada contraparte externa admite (HMAC propio para Original Auth y para las llamadas salientes/entrantes propias de Original Pay; credenciales en texto dentro del cuerpo

para el webhook de Original Pay, por diseño de ese proveedor).

6. Realtime

Diez tablas están habilitadas para suscripción en vivo (`publicaciones`, `comments`, `iniciativas`, `votaciones`, `beneficios`, `servicios`, `hc_transactions`, `personal_profiles`, `votacion_comments`, `saved_items`), seleccionadas específicamente porque cada una respalda una pantalla que se beneficia de actualización sin recarga manual (el feed, el saldo de HC, el catálogo de beneficios activos, entre otros). El resto del esquema se consulta bajo demanda, evitando el costo de mantener canales de suscripción innecesarios.

7. Integración con MediaAAS

Único servicio externo integrado desde el inicio del proyecto (previo a Original Auth y Original Pay), consumido exclusivamente a través de la Edge Function `upload-media`. De los seis endpoints documentados de MediaAAS, **HeartCoin utiliza únicamente uno** (`/client/upload`) — el resto (listado de archivos, URLs firmadas, gestión de carpetas, actualización y borrado de metadatos) están disponibles pero deliberadamente no integrados, reflejando un principio de exposición mínima: solo se construye la superficie de integración estrictamente necesaria para las funcionalidades existentes.

Problemas Detectados

1. **Ausencia de una especificación formal de API** (tipo OpenAPI/Swagger): la documentación existente (`ENDPOINTS.md` y este documento) es de naturaleza descriptiva y mantenida manualmente, sin un contrato machine-readable que permita generación automática de clientes o validación automatizada de contratos.
2. **Heterogeneidad en los mecanismos de autenticación de las Edge Functions**: conviven HMAC con nonce (Original Auth, Original Pay saliente/entrante propio) y credenciales en texto plano dentro del cuerpo (webhook de Original Pay), lo cual, si bien está justificado caso por caso, exige que cualquier desarrollador nuevo comprenda tres patrones distintos en lugar de uno solo.
3. **`personal_profiles` expone una policy de lectura pública a nivel de fila, no de columna**: cualquier usuario autenticado con acceso a un perfil marcado como público puede, técnicamente, solicitar cualquier columna de ese registro vía la API —incluyendo columnas que la aplicación oficial nunca solicita—, dado que RLS en PostgreSQL no restringe por columna de forma nativa.
4. **Ausencia de límites de tasa (rate limiting) documentados** a nivel de las Edge Functions propias del proyecto, más allá de los que Supabase pueda aplicar de forma genérica a nivel de plataforma.

Riesgos e Impactos

- El riesgo del punto 1 es de **impacto en mantenibilidad**: a medida que crece el número de integraciones externas, la ausencia de un contrato formal incrementa el costo de onboarding de

nuevos desarrolladores y el riesgo de documentación desactualizada respecto al código real.

- El riesgo del punto 3 es de **impacto en confidencialidad de bajo alcance**: las columnas actualmente presentes en `personal_profiles` no incluyen datos altamente sensibles más allá de los ya expuestos intencionalmente (nombre, ciudad, avatar, bio), pero el patrón representa una superficie de exposición mayor a la estrictamente necesaria y debe revisarse ante la incorporación de nuevas columnas sensibles a esa tabla.
- El riesgo del punto 4 es de **impacto en disponibilidad**: sin límites de tasa explícitos, las Edge Functions —particularmente las que realizan escritura financiera (`originalpay-redeem`)— podrían ser objeto de abuso por volumen de peticiones, aunque mitigado parcialmente por la exigencia de firma HMAC válida en la mayoría de los casos.

Fortalezas Identificadas

- **Correspondencia consistente entre sensibilidad de negocio y nivel de restricción de acceso**, verificable tabla por tabla en todo el esquema.
- **Contratos de error nombrados y explícitos** en las funciones RPC, superior a la propagación de errores de base de datos sin traducir.
- **Principio de exposición mínima aplicado de forma disciplinada**, evidenciado tanto en la integración parcial y deliberada de MediaAAS como en el reducido número de Edge Functions, cada una justificada por un requisito de seguridad concreto.
- **Verificación funcional activa** de los endpoints de integración externa más recientes, superando la validación puramente documental.

Áreas de Mejora

- Adoptar una especificación formal de API (OpenAPI) al menos para las Edge Functions propias, dado que son la porción de la superficie de API sobre la que el equipo tiene control total de diseño.
- Evaluar la incorporación de una vista (`view`) o función RPC específica para exponer únicamente las columnas públicas de `personal_profiles`, en lugar de depender de que el cliente oficial simplemente "no pida" las demás columnas.
- Documentar explícitamente cualquier límite de tasa existente a nivel de plataforma Supabase, y evaluar la necesidad de límites adicionales a nivel de aplicación para los endpoints financieros.

Recomendaciones

1. Priorizar la corrección de la exposición de columnas en `personal_profiles` (punto 3) mediante una vista pública restringida, dado que es el hallazgo de mayor sensibilidad de este documento.

2. Mantener el patrón de contratos de error nombrados (ya presente en las funciones RPC) como estándar obligatorio para toda nueva Edge Function.
3. Evaluar la generación de documentación de API semi-automatizada a partir del código de las Edge Functions, para reducir el riesgo de desactualización manual observado históricamente en este proyecto.

Conclusión

La superficie de API de HeartCoin, aunque no diseñada bajo una especificación formal centralizada, exhibe un nivel de coherencia y disciplina de seguridad notable para un proyecto de su escala: cada mecanismo de exposición —autogenerado o personalizado— sigue un patrón identificable de restricción de acceso correspondiente a la sensibilidad del dato o la operación involucrada. Las áreas de mejora identificadas son de naturaleza incremental, orientadas principalmente a formalizar y documentar patrones que ya existen en la práctica, más que a corregir debilidades estructurales del diseño.

Anexos

A. Resumen cuantitativo de la superficie de API

Categoría	Cantidad
Tablas expuestas vía PostgREST	20
Funciones RPC (<code>SECURITY DEFINER</code>)	3
Edge Functions	8
Tablas habilitadas para Realtime	10
Servicios externos integrados	3 (MediaAAS, Original Auth, Original Pay)

B. Documentación de referencia complementaria

- `ENDPOINTS.md` — catálogo línea por línea de cada tabla, policy y endpoint.
- `ORIGINAL_AUTH_INTEGRATION.md` / `ORIGINAL_PAY_INTEGRATION.md` — diseño detallado de cada integración externa.
- `mediaas-client-api-guide.md` — especificación completa del servicio externo MediaAAS.